Approach and Design Decisions

Personal AI Knowledge Assistant using Retrieval-Augmented Generation (RAG)
A grounded, document-based question-answering system.

1. The problem

Build an assistant that answers questions about a user's own documents, in natural language, without making facts up. The hard part is not generating text. Any large language model can do that. The hard part is keeping the answers grounded in the uploaded documents and being able to prove it.

2. The approach: Retrieval-Augmented Generation

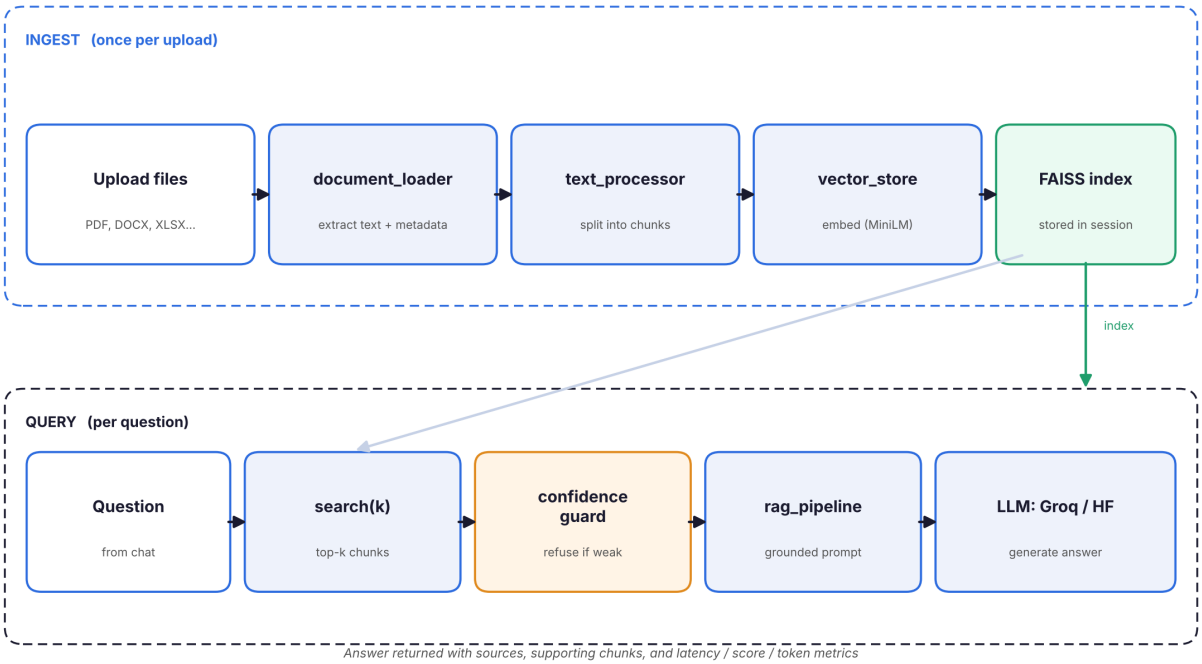
A plain language model answers from what it memorized during training. That is exactly what we do not want, because it has never seen the user's documents and will invent plausible-sounding answers. RAG fixes this by adding a retrieval step before generation:

1. Index the user's documents into a searchable vector database.
2. For each question, retrieve only the most relevant passages.
3. Give those passages to the model as context.
4. The model answers from that context, not from memory.

This keeps answers tied to real source material and lets us cite exactly where each answer came from. The system runs in two phases: **Ingest** (once per document set: load, chunk, embed, index) and **Query** (per question: retrieve, guard, prompt, generate).

3. Architecture

The system has two paths. **Ingest** runs once per document set and turns files into a searchable index. **Query** runs for every question: it retrieves the most relevant chunks, applies the confidence guard, and only then calls the LLM.



4. Design decisions

Each decision lists what we chose, why, and the tradeoff we accepted.

4.1 RAG instead of fine-tuning

Fine-tuning would be slow, expensive, and need retraining whenever a document changes. RAG needs no training. New documents are indexed in seconds. Tradeoff: answer quality depends on retrieval quality, so retrieval is where we spend our attention.

4.2 Modular architecture

The pipeline is split into four independent modules: document loader, text processor, vector store, and RAG pipeline. Each does one job and can be tested on its own, which keeps the code readable and easy to debug.

4.3 Multi-format loader with citation metadata

The loader supports 13 file types, where most projects support PDF only. Every passage carries its source filename and page, slide, or sheet number. That metadata is what makes source citations possible. Tradeoff: scanned image PDFs and the legacy .doc format are not supported, as they need OCR or system-level tools that complicate deployment.

4.4 Chunking: 800 characters, 150 overlap

Documents are split into roughly 800-character chunks that overlap by 150, breaking on paragraphs first. Embeddings work best on focused passages, and the overlap stops a boundary sentence from losing context. Tradeoff: too large and retrieval gets vague, too small and context is lost. These are the main tuning knobs.

4.5 Embeddings: all-MiniLM-L6-v2

Chunks are embedded with a small Sentence Transformer that is fast on CPU, about 90 MB, and strong for general use. It runs locally with no API cost. A larger model would be slightly more accurate but slower, which is not worth it at this scale.

4.6 Vector store: FAISS with cosine similarity

Embeddings are normalized and stored in a FAISS inner-product index, so the score is a clean cosine value. FAISS is fast and gives exact, fully accurate search at this scale. The clean score also powers the confidence guard and the on-screen metrics. The index saves and reloads, so documents are embedded once. Tradeoff: exact search would slow at millions of chunks, where an approximate index would be used instead.

4.7 LLM provider: pluggable, Groq by default

Generation sits behind a single function, so the provider swaps in one line. Groq is the default, with Hugging Face as a fallback. Hugging Face's free inference became paywalled during development, so we designed the layer to be provider-agnostic rather than locked to one vendor. Groq is free, fast, and reliable for a live demo.

4.8 Prompt engineering: grounding by construction

The prompt combines a strict system instruction, recent chat history, and the retrieved context plus the question. The system instruction tells the model to answer only from the context and to say so when the answer is not there. This is the first layer of hallucination defense, and the chat history gives the assistant memory for follow-up questions.

4.9 Confidence guard: the hard wall against hallucination

Before the model is called, the top retrieval score is checked against a threshold. If the best match is too weak, the app refuses and the model is never invoked. A prompt is a soft instruction a confident model can ignore. The guard is a hard rule in code that it cannot. Together they form a two-layer defense. This is the single most important choice for the hallucination metric, and it is directly demonstrable: an off-topic question scores far below the threshold and gets refused.

4.10 Transparency and metrics

Every answer shows its latency, top score, chunks used, and token count, plus an expandable view of the exact chunks behind it. A separate evaluation script measures retrieval accuracy, latency, and guard behavior across test questions. Most projects claim their system works. This one shows the numbers.

4.11 Configuration via environment variables

API keys load from a local .env file, and the same variable names are read from Streamlit Cloud secrets on deployment. No secrets in the code, and the same code runs locally and in the cloud.

5. How the design maps to the evaluation metrics

Metric	How the design addresses it
Retrieval accuracy	FAISS cosine search over overlapping chunks; measured as top-k hit rate, and the score is shown live.
Response quality	Grounded prompt plus exact-quote instruction; sanity-checked by keyword match in the evaluation script.
Hallucination rate	Two-layer defense: strict prompt plus the confidence guard that refuses weak-retrieval questions before the model runs.
Latency	Measured and displayed per answer, and reported as average and p95 by the evaluation script.
User experience	Clean chat UI with memory, source citations, retrieval transparency, and inline metrics.

6. Known limitations

- Scanned or image-only PDFs are not read, as there is no OCR step.
- The legacy binary .doc format is not supported.
- Exact FAISS search is ideal here but would need an approximate index at very large scale.
- The keyword-match quality check is a rough proxy, not a true measure of correctness.
- Free LLM tiers have rate limits, which can affect a long demo session.

7. Possible future improvements

- Reranking: retrieve more chunks, then reorder with a cross-encoder for better precision.
- Streaming answers: show text token by token for a more responsive feel.
- OCR support: read scanned PDFs and images.
- Hybrid search: combine keyword and vector search for better recall.

The key message: this is not just a chatbot. It is a grounded, measurable system that proves its answers come from your documents and refuses to guess when they do not.